SIH 2026 — General Project Documentation

Adaptive Career & Skill Trainer | Technical Reference v1.0

> **Document Status:** Stage 1 — General System Documentation
> **Audience:** All six team members
> **Purpose:** Authoritative technical reference for the entire project
> **Scope:** System-level only. Individual role documents follow separately.

Table of Contents

---

1. Project Objective and Problem Definition

1.1 Problem Statement Reference

SIH Problem Statement ID: 26044
Title: *Portal for Academia–Industry Collaboration for Skill Mapping, Internships and Placement*

1.2 Official Problem Summary

The official statement identifies a structural gap between what students learn in academic programs and what industry expects from new entrants. Students struggle at four specific points:

1. Identifying a career direction appropriate for their interests and abilities
2. Understanding which skills that career requires and at what proficiency level
3. Developing those skills through a structured, personalized pathway
4. Finding and qualifying for relevant opportunities such as internships, hackathons, and placements

1.3 Our Interpretation

The problem is not that students lack access to resources. Internshala, Unstop, SWAYAM,

NPTEL, AICTE's internship portal, and dozens of other platforms already provide courses, internships, assessments, and competitions. The problem is that these resources are:

- **Disconnected** from each other and from the student's current position
- **Undifferentiated** — the same opportunities are shown regardless of a student's readiness
- **Passive** — they require the student to already know what to look for
- **Unpersonalized** — there is no system that models an individual student's evolving skill state and adjusts guidance accordingly

Our interpretation of the deeper problem:

> **Students have access to many career resources but lack a personalized system that tells them what direction to pursue, what they currently know, what they should learn next, how their path should adapt as they progress, and which opportunities are appropriate for their current readiness.**

We are therefore not building another resource marketplace. We are building the **intelligence and progression layer** that connects those resources to individual students in a personalized, structured, and explainable way.

1.4 What We Are Building

An **adaptive career-readiness platform** that:

- Constructs a standardized career and skill graph modeling career domains, roles, competencies, skills, and their relationships
- Models an individual student's evolving proficiency across that graph
- Dynamically personalizes a learning and development trajectory for each student
- Aggregates external opportunities from existing platforms according to the student's current readiness
- Explains every recommendation in terms the student can act on

1.5 What We Are Not Building

Excluded from scope	Reason
A job/internship marketplace	Internshala, Unstop, and AICTE already do this

Excluded from scope	Reason
A course platform or LMS	SWAYAM, NPTEL already provide this
A social or professional network	LinkedIn and others already do this
A recruitment ATS	Not the student-facing problem
A complete college ERP	Beyond scope
A production-grade nationwide scraping infrastructure	Prototype does not require this
Sophisticated ML research models	Core value is the graph + personalization, not ML novelty

---

2. Product Vision

2.1 One-sentence description

> **An adaptive career-readiness platform that builds a standardized career and skill graph, models an individual student's evolving proficiency, dynamically personalizes their learning trajectory, and aggregates external opportunities according to their current readiness.**

2.2 Differentiation from existing platforms

Existing platforms primarily **provide** resources and opportunities. Our system **decides** how those resources fit into an individual student's progression. The distinction is:

Existing platforms	Our system
Show available opportunities	Show appropriate opportunities for this student right now
Offer a course catalog	Tell the student which specific skill to address next and why

Existing platforms	Our system
Provide static career guides	Maintain a live model of the student's skill state
Require the student to self-navigate	Guide the student through a structured, adaptive trajectory

2.3 The Core Loop

The fundamental product loop is:

```
Discover → Decide → Develop → Demonstrate → Opportunity
```

Phase	What happens
Discover	Student explores career domains informed by industry demand. System explains what each domain involves.
Decide	Student selects a career path. System maps it to a skill graph and possible technology branches.
Develop	System assesses current skills and generates a personalized roadmap. Student follows it.
Demonstrate	Assessments and optional evidence verify progress. Skill profile updates.
Opportunity	System surfaces relevant internships, hackathons, and projects matched to current readiness.

2.4 The Student Dashboard

The student's primary interface should make their current state immediately readable:

```
Your Goal: Backend Developer

Current Readiness          ██████▒▒ 72%

Current Focus              Spring Boot → REST APIs

Skills to Strengthen       SQL (Average) · Testing (Beginner) · Docker (Not started)

Recommended Next Step      → REST API Assessment

Opportunities You Can Target Now
→ 3 Backend Internships    → 2 Hackathons

Opportunities You're Close To
→ 2 Internships (need: Docker basics)
```

---

3. Functional Scope

3.1 In-scope for prototype

Feature area	Specific functionality
Student profile	Registration, education details, interests, selected career, current skills, proficiency levels, progress tracking
Career discovery	Industry-demand-informed domain listing, domain explanations, career path overview, technology branch visualization, curated external resources
Career and skill graph	Stored hierarchy of domains, roles, competencies, skills, technology branches, prerequisites
Self-assessment	Structured self-rating against career-relevant skills
Targeted	System-generated questions calibrated to career and self-rating; scoring and

Feature area	Specific functionality
assessment	proficiency update
Adaptive roadmap	Baseline path per role, predefined decision/branch points, student choice at decision points, dynamic recalculation of remaining roadmap
Progress tracking	Milestone completion, skill advancement, readiness percentage
Opportunity index	Scraped and manually seeded opportunities from Unstop, Internshala, AICTE; normalized, skill-tagged, deduplicated
Opportunity matching	Deterministic compatibility scoring between student profile and indexed opportunities; ranked, explained results
Evidence portfolio	Optional student submission of projects, GitHub links, certifications as supplementary signals
AI assistance	Skill extraction from job postings, personalized explanations, assessment question generation, semantic matching assistance

3.2 Out-of-scope for prototype (future scope)

Explicitly not built now. Documented in Section 28.

---

4. Non-Functional Requirements

4.1 Performance

Metric	Prototype target	Rationale
API response time (p95)	< 800 ms for standard queries	SIH demo must be responsive
Assessment load	< 1 second	Poor UX kills demo credibility

Metric	Prototype target	Rationale
time		
Opportunity matching	< 2 seconds	Acceptable for recommendation workload
AI-assisted responses	< 5 seconds with loading indicator	LLM latency is inherently higher
Page load (frontend)	< 3 seconds on average connection	Standard web expectation

> **Prototype note:** These are demonstration targets, not production SLAs. The system does not need to handle concurrent load beyond a small demo session.

4.2 Scalability

> **Prototype:** Single-server deployment. No horizontal scaling infrastructure required.
> **Production:** Stateless API design enables horizontal scaling. Scraping pipeline and recommendation engine become independently scalable services.

4.3 Reliability

Requirement	Prototype approach
API uptime	Best-effort for demo; no on-call or SLA
Data loss prevention	Regular database backups during development
Scraping failures	Fallback to manually seeded data; errors logged
AI failures	Graceful degradation to deterministic fallback

4.4 Usability

- Every recommendation must be accompanied by a human-readable explanation
- The student must always be able to see their current position in the journey
- No dead ends — every screen must offer a next step

- The skill/career graph must be visually browsable, not just read from tables

4.5 Security

Covered in detail in Section 22. Summary:

- JWT-based authentication
- Role-based access control (student vs. admin)
- No storage of plaintext passwords
- Input validation and sanitization on all endpoints
- Scraped data treated as untrusted input

4.6 Privacy

- Student skill data is private to the student
- No student data shared externally without consent
- Prototype does not integrate with any third-party analytics trackers
- Scraped opportunity data must not include personal data from third-party platforms

4.7 Maintainability

- Each module has a single responsible owner (see Section 6)
- API contracts documented and versioned
- No undocumented magic in the recommendation or AI layers
- Skill graph data stored in structured, queryable form — not embedded in application code

4.8 Accessibility

> **Decision Pending** — Full WCAG compliance is aspirational for the prototype. Recommended default: semantic HTML, sufficient color contrast, keyboard navigation on primary flows. Full accessibility audit is post-prototype scope.

---

5. Complete End-to-End User Journey

This section documents every major step a student takes from first visit to opportunity matching.

This journey is the **golden demonstration path** that the prototype must implement flawlessly.

5.1 Journey Map

```
[1] Land on platform
        ↓
[2] Register / Onboard
        ↓
[3] Career Discovery
        ↓
[4] Career Selection
        ↓
[5] Self-Assessment
        ↓
[6] Targeted Assessment
        ↓
[7] Skill Profile Generated
        ↓
[8] Adaptive Roadmap Generated
        ↓
[9] Student Progresses (learns, completes milestones)
        ↓
[10] Optional Evidence Submission
        ↓
[11] Opportunity Matching
        ↓
[12] Student Reviews Opportunities
        ↓
[13] Student Redirected to External Source
```

5.2 Detailed Step Descriptions

Step 1 — Landing

The student arrives at the platform. The landing page explains the product concept briefly and invites registration. No functionality is accessible without an account.

Step 2 — Registration and Onboarding

The student:

1. Creates an account (name, email, password, institution, degree, year of study)
2. Answers a short onboarding questionnaire:
    - Broad interest areas (multiple selection, not exclusive)
    - Self-described experience level (total beginner / some background / experienced)
3. Profile is created. Student is directed to Career Discovery.

Step 3 — Career Discovery

The system displays a set of career domains. For the prototype, these are a curated set covering approximately 6–10 domains (e.g., Software Engineering, Data and AI, Cybersecurity, UI/UX and Product, Cloud and DevOps, Embedded Systems).

Each domain card displays:

- Domain name
- One-paragraph plain-language description of what professionals in this domain actually do
- 3–5 relevant technologies or tools
- Current demand indicator (high / moderate / emerging) derived from scraped job data
- A "Explore" action

When a student explores a domain, they see:

- Detailed explanation of the domain
- Career paths within it (roles)
- Example companies and real-world applications
- An industry-demand snapshot (top required skills by frequency from scraped postings)
- A curated set of 3–5 external resources to understand the domain

The student can explore multiple domains. They are not forced to commit immediately.

> **Design principle:** A student cannot meaningfully choose between cybersecurity and data engineering if they do not understand what either involves. Exploration must come before selection.

Step 4 — Career Selection

The student selects one primary career path for now. They choose:

1. Domain (e.g., Software Engineering)
2. Role (e.g., Backend Developer)
3. Technology branch if applicable (e.g., Python / Node.js / Java)

The system visualizes this as a branch in the skill graph. The student can see the selected branch clearly alongside alternatives they chose not to pursue.

Step 5 — Self-Assessment

The system presents the student with the core competencies and skills associated with their chosen role.

For each skill, the student rates their proficiency:

- Not familiar
- Beginner (heard of it, understand conceptually)
- Average (can use with reference/documentation)
- Proficient (can use independently)
- Excellent (can teach it)

The system does not yet judge these ratings. It records them as **student perception**.

Step 6 — Targeted Assessment

Based on the selected role and the self-assessment, the system generates a targeted assessment. The purpose is to **calibrate** the student's self-perception against actual performance.

The assessment:

- Covers skills the student rated Average or above (to detect overestimation)
- Includes foundational skills regardless of rating (prerequisites must be verified)
- Is approximately 20–30 questions per session for the prototype
- Covers conceptual understanding, not full coding interviews

After submission, the system computes a proficiency score per skill. It compares this to the self-assessment and surfaces discrepancies without labeling them negatively:

```
Skill: SQL
Self-assessment: Average
Assessment result: Beginner

→ "Your assessment suggests SQL may need more attention than you expected.
   We've adjusted your roadmap accordingly."
```

Step 7 — Skill Profile Generated

A structured skill profile is produced containing:

- Each relevant skill
- Assessed proficiency level
- Confidence indicator (high confidence if self-rating matched assessment; lower if discrepant)
- Gap relative to role target proficiency

This profile is visible to the student as their Skill Dashboard.

Step 8 — Adaptive Roadmap Generated

Using the skill profile, the system generates a personalized roadmap:

- Skills already at target proficiency → marked complete or optional deepening
- Skills below target → ordered by prerequisite dependency and priority
- The roadmap has explicit **decision points** where the student can choose a branch

The roadmap is not a flat to-do list. It reflects the prerequisite structure of the skill graph. A student cannot be directed to Spring Boot before REST API fundamentals are established.

Decision points example:

```
REST APIs (required)
     ↓
Choose database focus:
  → [SQL / PostgreSQL]  OR  → [NoSQL / MongoDB]
```

The student chooses. The remaining roadmap recalculates.

Step 9 — Student Progresses

The student follows the roadmap. As they complete:

- Assessments at each skill milestone
- Optional evidence submission

The skill profile updates. Readiness percentage increases. The opportunity panel updates accordingly.

Step 10 — Optional Evidence Submission

The student can optionally submit:

- GitHub repository links
- Certificates (link or upload)
- Project descriptions
- Hackathon/competition participation

The system records these as **supplementary confidence boosters**. They are not required. A student without a GitHub account is not penalized. Evidence strengthens the profile but the assessment score is the primary signal.

Step 11 — Opportunity Matching

At any point from profile creation onward, the student can view their matched opportunities. The system:

1. Retrieves the student's current skill profile and readiness
2. Queries the Opportunity Index
3. Applies the compatibility scoring algorithm
4. Sorts and segments results into:
    - **Ready now** — student meets or exceeds stated requirements
    - **Almost ready** — student meets most requirements; 1–2 skills are gaps
    - **Aspirational** — good career alignment but significant skill gaps remain

Each result shows:

- Title, organization, type (internship / hackathon / project)
- Compatibility score

- Explanation: which skills matched, which are gaps
- Original source link (redirects to Internshala, Unstop, AICTE, etc.)

Step 12 — Student Reviews Opportunities

Student browses matched opportunities. They can:

- Filter by type, mode (remote/on-site), deadline
- View detailed match explanation
- Save opportunities to a watchlist
- Mark as applied (no tracking beyond this)

Step 13 — Redirect to External Source

The student is redirected to the original opportunity URL on the source platform. We do not host application forms. The platform's role ends at intelligent matching and explanation.

---

6. System Architecture

6.1 Architectural Philosophy

The prototype uses a **modular monolith** architecture: one backend application organized into well-separated domain modules, communicating internally through defined service interfaces rather than network calls.

Reason this was chosen over microservices:

Microservices add significant operational overhead (service discovery, inter-service networking, distributed tracing, separate deployments) that provides no benefit during a prototype with a small team and controlled demo environment. The value of microservices is operational independence at scale. We do not yet have that requirement.

The module boundaries we establish now are **designed to be extracted into independent services later** if needed. The key discipline is that modules do not directly access each other's database tables — all cross-module interaction goes through service interfaces.

6.2 High-Level Architecture Diagram

graph TB subgraph Client Layer FE[Frontend
React SPA] end subgraph API Gateway Layer AG[API Gateway
Routing · Auth · Rate Limiting] end subgraph Application Layer AUTH[Auth Module] PROFILE[Profile Module] CAREER[Career Module] SKILL[Skill Graph Module] ASSESS[Assessment Module] ROADMAP[Roadmap Module] OPPS[Opportunity Module] REC[Recommendation Module] AI[AI Module] RESOURCE[Resource Module] PORTFOLIO[Portfolio/Evidence Module] end subgraph Data Layer PG[(PostgreSQL
Primary Database)] CACHE[(Redis Cache
Sessions · Frequent reads)] FS[File Storage
Evidence uploads] end subgraph Background Layer PIPE[Data Pipeline
Scraper · Parser · Normalizer] SCHED[Scheduler
Periodic scrape runs] end subgraph External Layer UNSTOP[Unstop] INSHALA[Internshala] AICTE[AICTE Portal] LLM[LLM API
OpenAI / Gemini] end FE --> AG AG --> AUTH AG --> PROFILE AG --> CAREER AG --> SKILL AG --> ASSESS AG --> ROADMAP AG --> OPPS AG --> REC AG --> RESOURCE AG --> PORTFOLIO AUTH --> PG PROFILE --> PG CAREER --> PG SKILL --> PG ASSESS --> PG ROADMAP --> PG OPPS --> PG REC --> PG RESOURCE --> PG PORTFOLIO --> PG PORTFOLIO --> FS REC --> AI ASSESS --> AI AI --> LLM AG --> CACHE AUTH --> CACHE SCHED --> PIPE PIPE --> UNSTOP PIPE --> INSHALA PIPE --> AICTE PIPE --> PG

6.3 Component Ownership Summary

Component	Owner Role
Frontend (all pages)	Role 1 — Frontend
API Gateway, Auth, Profile, Core Backend	Role 2 — Backend
Career Module, Skill Graph Module, Assessment Module, Roadmap Module	Role 3 — Career & Skill Graph
Data Pipeline, Opportunity Module	Role 4 — Data Engineering
Recommendation Module, AI Module	Role 5 — AI & Recommendation
System integration, deployment, cross-cutting concerns	Role 6 — Architecture & Integration

> **Note on Role 6:** Six team members are defined in the blueprint. The sixth workstream is

> System Architecture and Integration, which ensures the five parallel workstreams produce one coherent system. This role may be held by the strongest backend/full-stack member.

---

7. Major System Components

7.1 Frontend Application

A single-page React application implementing all student-facing screens. Communicates exclusively through the Backend API. Contains no business logic — all decisions are made server-side.

7.2 Backend API

The central application. A single deployable service organized into domain modules. Owns all business logic, data validation, authentication, and routing. All frontend requests pass through this.

7.3 Career Module

Manages the static domain-and-role hierarchy: what careers exist, what they involve, how they are described.

7.4 Skill Graph Module

Manages the detailed knowledge model: competencies, skills, technology branches, prerequisites, and relationships. This is the **most conceptually critical component** in the system.

7.5 Assessment Module

Manages question banks, assessment session creation and delivery, answer submission, and scoring. Produces proficiency scores per skill.

7.6 Roadmap Module

Manages baseline career paths, decision points, student path state, prerequisite enforcement, and dynamic roadmap recalculation.

7.7 Profile Module

Manages student personal data, education, selected career, aggregated skill state, and progress.

7.8 Opportunity Module

Manages the Opportunity Index — the normalized, deduplicated, skill-tagged collection of opportunities collected from external sources.

7.9 Recommendation Module

Produces ranked, explained recommendations by combining student profile data with the Opportunity Index using deterministic scoring supplemented by AI-assisted semantic matching.

7.10 AI Module

Wraps all LLM interactions. Provides a controlled, validated interface so that no other module calls an LLM directly. AI Module is a service used by other modules.

7.11 Data Pipeline

An independent background process that scrapes, parses, normalizes, and indexes opportunities from external sources. Writes to the same database. Does not participate in request handling.

7.12 Resource Module

Manages curated learning resources linked to specific skills or roadmap milestones. Resources are externally hosted (SWAYAM, YouTube, documentation). The module stores links and metadata only.

7.13 Portfolio/Evidence Module

Manages optional student evidence submissions: GitHub links, certificates, project descriptions.

---

8. Component Responsibilities and Boundaries

8.1 Explicit boundary rules

These rules prevent duplication and confusion between modules:

Rule	Description
Modules do not read each other's tables	Cross-module data access is through service method calls only
AI Module is the sole LLM caller	No other module imports or calls the LLM API directly
Data Pipeline is the sole opportunity writer	The Opportunity Module reads; only the Pipeline writes raw opportunity data
Skill Graph Module is the sole skill taxonomy authority	Other modules reference skill IDs but do not maintain skill definitions
Assessment Module is the sole scorer	Other modules receive scores; they do not recompute them
Recommendation Module is the sole ranker	Other modules do not produce ranked opportunity lists

8.2 Module boundary table

Module	Owns	Does not own
Career Module	Domain list, role list, domain-role mapping	Skill definitions, prerequisite logic
Skill Graph Module	Skill nodes, edges (prerequisites), competencies, technology branches	Career descriptions, assessment questions
Assessment Module	Questions, sessions, answers, scores	Skill taxonomy, roadmap state

Module	Owns	Does not own
Roadmap Module	Path templates, decision points, student path state	Assessment execution, opportunity data
Profile Module	Student record, aggregated skill profile	Raw assessment results (links to Assessment Module)
Opportunity Module	Indexed opportunity records	Scraping logic, student matching
Recommendation Module	Ranked results, match explanations	Opportunity storage, student profile storage
AI Module	LLM calls, prompt management, output validation	Any domain data, direct DB access
Data Pipeline	Scraping, parsing, normalization, writing to Opportunity table	Serving API requests
Resource Module	Resource records linked to skills/ milestones	Skill taxonomy
Portfolio Module	Evidence records, links to student profile	Scoring, assessment logic

---

9. Data Flow Between Components

9.1 Student onboarding flow

sequenceDiagram participant S as Student (Browser) participant API as Backend API participant AUTH as Auth Module participant PROF as Profile Module participant CAR as Career Module S->>API: POST /auth/register API->>AUTH: Create user record AUTH-->>API: User created, JWT issued API-->>S: JWT token S->>API: GET /careers/domains (with JWT) API->>CAR: Fetch domain list with demand signals CAR-->>API: Domain list API-->>S: Domain list with explanations S->>API: POST /profile/career-selection API->>PROF: Store career selection API->>CAR: Fetch skill graph for role CAR-->>API: Skill graph nodes API-->>S: Career selected, ready for

assessment

9.2 Assessment flow

sequenceDiagram participant S as Student participant API as Backend API participant ASSESS as Assessment Module participant AI as AI Module participant PROF as Profile Module S->>API: POST /assessment/start (self-ratings included) API->>ASSESS: Create assessment session ASSESS->>AI: Generate question set (career + self-ratings) AI-->>ASSESS: Question set ASSESS-->>API: Session ID + questions API-->>S: Questions S->>API: POST /assessment/submit (answers) API->>ASSESS: Score answers ASSESS-->>API: Per-skill scores API->>PROF: Update skill profile API-->>S: Results + updated profile

9.3 Roadmap generation flow

sequenceDiagram participant API as Backend API participant ROAD as Roadmap Module participant SKILL as Skill Graph Module participant PROF as Profile Module API->>ROAD: Generate roadmap (student_id) ROAD->>PROF: Get student skill profile PROF-->>ROAD: Skill states ROAD->>SKILL: Get prerequisite graph for role SKILL-->>ROAD: Ordered prerequisite graph ROAD->>ROAD: Compute remaining skills
Identify decision points
Order by dependency ROAD-->>API: Personalized roadmap

9.4 Opportunity recommendation flow

sequenceDiagram participant S as Student participant API as Backend API participant REC as Recommendation Module participant OPP as Opportunity Module participant PROF as Profile Module participant AI as AI Module S->>API: GET /recommendations/opportunities API->>REC: Get recommendations (student_id) REC->>PROF: Get skill profile + career goal REC->>OPP: Get candidate opportunities (filtered by career domain) REC->>REC: Apply deterministic compatibility score REC->>AI: Semantic matching for borderline cases AI-->>REC: Semantic scores REC->>REC: Merge scores, rank, segment REC-->>API: Ranked opportunities with explanations API-->>S: Opportunity list

9.5 Data pipeline flow (background)

graph LR SCHED[Scheduler] -->|trigger| SCRAPE[Scraper] SCRAPE -->|raw HTML/JSON| PARSE[Parser] PARSE -->|structured records| NORM[Normalizer] NORM -->|normalized

records| DEDUP[Deduplicator] DEDUP -->|check existing| DB[(Opportunity Table)] DEDUP -->|new/updated records| SKILL_EX[Skill Extractor] SKILL_EX -->|AI-assisted| AI[AI Module] AI -->|extracted skill tags| SKILL_EX SKILL_EX -->|tagged records| DB

---

10. Core Domain Model

10.1 Entity relationship overview

erDiagram USER ||--|| STUDENT_PROFILE : has STUDENT_PROFILE ||--o{ SKILL_STATE : contains STUDENT_PROFILE ||--|| ROADMAP_STATE : has STUDENT_PROFILE ||--o{ EVIDENCE_ITEM : submits STUDENT_PROFILE }o--|| ROLE : targets DOMAIN ||--o{ ROLE : contains ROLE ||--o{ COMPETENCY : requires COMPETENCY ||--o{ SKILL : contains SKILL ||--o{ SKILL : prerequisite_of SKILL ||--o{ TECHNOLOGY_BRANCH : has ASSESSMENT_SESSION }o--|| STUDENT_PROFILE : belongs_to ASSESSMENT_SESSION ||--o{ QUESTION_RESPONSE : contains QUESTION }o--|| SKILL : tests ROADMAP_STATE }o--|| ROLE : follows ROADMAP_STATE ||--o{ MILESTONE : contains MILESTONE }o--|| SKILL : covers OPPORTUNITY ||--o{ OPPORTUNITY_SKILL : requires SKILL ||--o{ OPPORTUNITY_SKILL : tagged_in OPPORTUNITY }o--|| DOMAIN : aligned_to RECOMMENDATION }o--|| STUDENT_PROFILE : for RECOMMENDATION }o--|| OPPORTUNITY : about

10.2 Primary entity list

Entity	Description	Owner module
`User`	Authentication identity	Auth
`StudentProfile`	Student personal and academic info	Profile
`SkillState`	Student's proficiency in a specific skill	Profile / Assessment
`Domain`	High-level career domain	Career
`Role`	Specific role within a domain	Career

Entity	Description	Owner module
`Competency`	Grouped cluster of skills for a role	Skill Graph
`Skill`	Atomic learnable unit	Skill Graph
`TechnologyBranch`	Specialization branch within a skill	Skill Graph
`SkillPrerequisite`	Directed prerequisite edge	Skill Graph
`AssessmentSession`	One instance of an assessment taken by a student	Assessment
`Question`	An individual assessment question	Assessment
`QuestionResponse`	Student answer to one question	Assessment
`RoadmapState`	Student's current position in their roadmap	Roadmap
`Milestone`	Ordered step in a roadmap	Roadmap
`DecisionPoint`	A branching choice in the roadmap	Roadmap
`Opportunity`	A normalized opportunity from any source	Opportunity
`OpportunitySkill`	Skill tag on an opportunity	Opportunity
`Recommendation`	A computed match between student and opportunity	Recommendation
`Resource`	An external learning resource linked to a skill	Resource
`EvidenceItem`	Student-submitted portfolio evidence	Portfolio

---

11. Career and Skill Graph Model

11.1 Purpose

The skill graph is the **intellectual foundation** of the entire system. Every other component — assessment, roadmap, recommendation, AI — depends on a well-structured, accurate,

prerequisite-aware knowledge model.

It is not a flat list of technologies. It is a directed acyclic graph (DAG) where:

- **Nodes** represent skills at different levels of specificity
- **Edges** represent prerequisite relationships (Skill A must precede Skill B)
- **Branches** represent alternate technology paths within the same competency
- **Roles** are defined by the specific set of competencies they require

11.2 Why a DAG matters

A flat list cannot answer the question: *"What should this student learn next?"*

A DAG can, because:

- It enforces that prerequisites are satisfied before advanced topics
- It enables the roadmap to automatically order skills by dependency
- It allows a student to choose a branch (Java vs Python) without invalidating the rest of their path
- It enables the recommendation engine to compute "how far is this student from being ready for this opportunity"

11.3 Hierarchy

```
Domain
  └── Role
        └── Competency
              └── Skill
                    ├── Technology Branch
                    └── (prerequisite edges to other Skills)
```

11.4 Example graph — Backend Developer (Python path)

```
Backend Developer
│
├── Competency: Programming Foundations
│     ├── Skill: Programming fundamentals (prerequisite: none)
│     ├── Skill: Python language (prerequisite: Programming fundamentals)
│     └── Skill: Object-Oriented Programming (prerequisite: Python language)
│
├── Competency: Web and API Development
│     ├── Skill: HTTP and the web (prerequisite: Programming fundamentals)
│     ├── Skill: REST API design (prerequisite: HTTP and the web)
│     └── Skill: FastAPI / Django REST (prerequisite: Python language + REST API design)
│
├── Competency: Data and Storage
│     ├── Skill: Relational databases (prerequisite: Programming fundamentals)
│     ├── Skill: SQL (prerequisite: Relational databases)
│     └── Skill: ORM usage (prerequisite: SQL + Python language)
│
├── Competency: Software Engineering Practices
│     ├── Skill: Version control (Git) (prerequisite: none)
│     ├── Skill: Testing fundamentals (prerequisite: Python language)
│     └── Skill: API testing (prerequisite: REST API design + Testing fundamentals)
│
└── Competency: Deployment and Operations
      ├── Skill: Linux basics (prerequisite: none)
      ├── Skill: Docker (prerequisite: Linux basics)
      └── Skill: Basic cloud deployment (prerequisite: Docker)
```

11.5 Technology branch model

A technology branch is a **specialization choice** at a specific skill node. Both paths within a branch are valid. The student chooses one. The prerequisite relationships of the chosen branch are then added to their roadmap.

```
REST API Development (required for all)
         ↓
   Framework choice (decision point)
   ├── FastAPI branch
   │     → Pydantic · Async Python
   └── Django REST branch
         → Django ORM · Django admin
```

11.6 Prototype data requirements

For the prototype, the skill graph must be manually curated and expert-reviewed. The system cannot operate without this data. The team must seed at least:

- 6–8 domains
- 3–5 roles per domain (or at least 2 per domain for prototype)
- Full competency/skill/prerequisite graph for at least 2 complete roles (the golden demo paths)
- Partial data for remaining roles

> **This is a content engineering task, not only a software engineering task.** Inaccurate or shallow skill graphs will make the entire system unconvincing.

11.7 Skill proficiency target levels

Each skill in a role has a **target proficiency level** — the level a student should reach to be "role-ready" for that skill.

Level	Description
`AWARENESS`	Knows the concept exists and its purpose
`BEGINNER`	Can use with significant reference/guidance
`INTERMEDIATE`	Can use independently on standard problems
`PROFICIENT`	Can use effectively including non-standard situations
`EXPERT`	Can design, teach, and evaluate

Not every skill needs `EXPERT` level. A backend developer needs SQL at `PROFICIENT`, while

Docker may only need `INTERMEDIATE` for an entry-level role.

---

12. Student Profile Model

12.1 Components

The student profile is a composite of several data sources:

Sub-profile	Source	Updated when
Identity	Registration	Account creation
Academic info	Onboarding	Onboarding; editable
Interests	Onboarding	Onboarding; editable
Career selection	User action	Career selection; changeable
Skill state	Assessment + Evidence	After each assessment; after evidence submission
Roadmap state	System-generated	After career selection; after assessments; after branch choices
Evidence portfolio	User submission	Any time

12.2 Skill state model

Each `SkillState` record for a student represents:

Field	Type	Description
`skill_id`	FK	Reference to Skill node
`self_rating`	Enum	Student's self-assessment rating

Field	Type	Description
`assessed_level`	Enum	System-determined proficiency after assessment
`confidence`	Enum (high/ medium/low)	Degree of agreement between self-rating and assessment
`evidence_boost`	Boolean	Whether evidence submission increased confidence
`last_assessed_at`	Timestamp	When last assessed
`assessment_score_pct`	Float	Raw percentage score on last relevant assessment

12.3 Readiness percentage

Readiness percentage is a **computed value**, not stored directly. It is recalculated on each request:

```
readiness = (
  Σ (assessed_level_score × skill_weight) for all required skills
) / (
  Σ (target_level_score × skill_weight) for all required skills
)
```

Where `skill_weight` reflects the relative importance of that skill within the role (defined in the skill graph). Skill weights for the prototype can be uniform (1.0) with differentiation as future scope.

---

13. Assessment Model

13.1 Three-layer design

Layer	Name	Purpose	Mandatory
1	Self-assessment	Capture student perception	Yes
2	Targeted assessment	Calibrate perception against actual performance	Yes
3	Evidence portfolio	Supplementary confidence signals	No

13.2 Question model

Each question in the bank belongs to:

- A specific `Skill`
- A specific `proficiency_level` it tests
- A `question_type` (MCQ / True-False / Scenario MCQ)
- Has metadata: `explanation`, `difficulty`, `time_estimate_seconds`

For the prototype:

- Questions are manually authored or AI-generated and human-reviewed
- All questions are multiple choice (simplest to auto-score)
- Open-ended and coding questions are future scope

13.3 Assessment session generation

Given:

- Student's selected role → determines relevant skills
- Student's self-ratings → determines question difficulty distribution
- Student's assessment history → avoids exact repeats

The session contains:

- Foundational prerequisite skills (always included regardless of self-rating)

- Skills rated Average or above → questions at Proficient level (to detect overestimation)
- Skills rated Beginner → questions at Beginner level (to establish baseline)
- Skills rated Not familiar → excluded from assessment (cannot be assessed without knowledge)

13.4 Scoring

Each session produces:

- Per-question: correct/incorrect, time taken
- Per-skill: percentage correct across questions testing that skill
- Per-skill: mapped proficiency level (scored percentage → proficiency enum)

Proficiency mapping (prototype default, adjustable):

Score range	Proficiency level assigned
0–20%	AWARENESS
21–45%	BEGINNER
46–65%	INTERMEDIATE
66–85%	PROFICIENT
86–100%	EXPERT

13.5 Discrepancy handling

Self-rating	Assessed level	System response
Excellent	Beginner or lower	Gentle recalibration message; roadmap includes foundational steps
Poor	Proficient or above	Positive acknowledgment; roadmap skips or fast-tracks early steps
Any	Matches self	Confirmation; normal roadmap

The system never accuses the student of dishonesty. Discrepancies are presented as information, not judgments.

---

14. Adaptive Roadmap Model

14.1 What the roadmap is

The roadmap is a personalized, ordered sequence of learning milestones derived from the skill graph. It is:

- **Prerequisite-ordered** — skills that depend on others always come after their prerequisites
- **Gap-filtered** — skills already at target proficiency are excluded or marked optional for deepening
- **Branch-aware** — at decision points, only the chosen branch's skills appear
- **Dynamic** — recalculated whenever the student's skill state changes or they make a branch choice

14.2 Baseline path

Every role has a **baseline path**: the default recommended ordering of skills and milestones, authored by the Career/Skill Graph team. This is the starting point before personalization.

14.3 Personalization process

```
Baseline path for role
        ↓
Filter out skills at target proficiency (already assessed)
        ↓
Reorder remaining skills by prerequisite graph topology (topological sort)
        ↓
Insert decision points at defined branch nodes
        ↓
Attach resources to each milestone
        ↓
Personalized roadmap emitted
```

14.4 Decision points

A decision point is a predefined node in the skill graph where the student must select a technology branch before proceeding. Decision points are not arbitrary — they are authored as part of the skill graph definition.

When a student reaches a decision point:

1. The system presents available branches with explanations
2. The student selects one
3. The roadmap module fetches the selected branch's skills
4. The remaining roadmap is recalculated and returned

14.5 Progress tracking

A milestone can be in one of the following states:

State	Meaning
`NOT_STARTED`	Student has not engaged with this skill yet
`IN_PROGRESS`	Student has started but not reached target proficiency
`COMPLETED`	Student has reached target proficiency
`SKIPPED`	Skill was at target proficiency at roadmap generation
`BRANCH_PENDING`	Awaiting student branch decision

---

15. Opportunity Aggregation and Indexing Model

15.1 Design principle

The system **does not host or own opportunities**. It indexes, normalizes, tags, and ranks externally hosted opportunities.

Every student-facing opportunity display includes:

- The original source platform name
- A direct link to the original opportunity
- Clear attribution

15.2 Sources (prototype)

Source	Type	Access method
Unstop	Internships, competitions, hackathons	Web scraping (prototype) / API (future)
Internshala	Internships, trainings	Web scraping (prototype)
AICTE Portal	Internships, apprenticeships	Web scraping / public listings (prototype)
Manually seeded	Any type	Team-curated seed data for demo reliability

> **Important:** Before scraping any source, the team must review that source's `robots.txt` file and terms of service. If scraping is restricted, the prototype must use manually seeded data for that source. This is a legal and ethical requirement, not a technical constraint.

15.3 Opportunity record structure

Field	Type	Description
`id`	UUID	Internal identifier
`external_id`	String	Source-specific identifier for deduplication
`source`	Enum	`UNSTOP`, `INTERNSHALA`, `AICTE`, `MANUAL`
`original_url`	URL	Canonical link to source
`title`	String	Opportunity title
`organization`	String	Offering organization
`type`	Enum	`INTERNSHIP`, `HACKATHON`, `COMPETITION`, `PROJECT`,

Field	Type	Description
		`TRAINING`
`mode`	Enum	`REMOTE`, `ON_SITE`, `HYBRID`, `UNSPECIFIED`
`location`	String	Physical location if applicable
`deadline`	Date	Application deadline
`duration`	String	Duration if specified
`stipend`	String	Stipend info if specified
`eligibility_raw`	Text	Raw eligibility text
`description_raw`	Text	Raw description
`required_skills`	String[]	Extracted skill labels
`skill_tags`	FK[]	Normalized skill IDs from skill graph
`domain_tags`	FK[]	Aligned domain IDs
`extracted_at`	Timestamp	When scraped/ingested
`last_seen_at`	Timestamp	Last confirmed alive
`is_active`	Boolean	Whether still open/valid

15.4 Skill extraction from opportunities

This is a critical step. Each opportunity's description and stated requirements are processed to:

1. Extract raw skill mentions (AI-assisted)
2. Normalize mentions to canonical skill graph nodes (e.g., "Spring" → `skill:spring-boot`)
3. Store both raw mentions and normalized tags

AI handles the fuzzy matching of raw mentions to taxonomy nodes. A human-authored normalization dictionary handles common cases deterministically. Unknown mentions are flagged for review.

15.5 Deduplication

Before inserting a scraped opportunity:

1. Compute a fingerprint: `hash(source + external_id)` if available, or `hash(title + organization + deadline)`
2. Check if fingerprint exists in the table
3. If exists → update `last_seen_at`, check for field changes, update accordingly
4. If new → insert

---

16. Recommendation Model

16.1 Philosophy

The recommendation system is **deterministic-first, AI-assisted for edge cases**. This means:

- The core compatibility score is computed by an explicit formula using structured data
- AI is used to improve scoring for cases where structured data is ambiguous (e.g., a vague job description that doesn't list skills explicitly)
- Every recommendation can be explained in human-readable terms derived from the formula inputs

This makes the system auditable and debuggable — important for a demo and important for student trust.

16.2 Compatibility score formula

```
compatibility_score = (
    skill_match_score     × 0.50
  + career_alignment_score × 0.25
  + eligibility_score      × 0.15
  + interest_score         × 0.10
)
```

Skill match score:

```
skill_match_score =
  Σ (min(student_level, required_level) / required_level × skill_weight)
  / Σ skill_weight
  for all required skills in the opportunity
```

If the student exceeds the required level for a skill, the score for that skill is capped at 1.0 (full match).

Career alignment score:
Binary or tiered: Does the opportunity's domain/role align with the student's selected career path? (Full match / partial match / no match)

Eligibility score:
Deterministic check: Does the student meet stated criteria such as year of study, institution type, or degree? Parsed from `eligibility_raw` where possible; otherwise defaults to 1.0 (assume eligible).

Interest score:
Does the opportunity's domain/type align with the student's stated interests from onboarding? Computed from interest tags on the student profile.

16.3 Segmentation

After scoring:

Segment	Threshold	Label
Ready now	≥ 0.75	"You're a strong match"
Almost ready	0.50–0.74	"Close match — missing 1–2 skills"
Aspirational	0.25–0.49	"A future target — keep progressing"
Not yet relevant	< 0.25	Not shown by default

16.4 Explanation generation

For each recommendation, the system generates a structured explanation:

```
{
  "summary": "Strong match — your Python and REST API skills align well.",
  "matching_skills": ["Python", "REST APIs", "SQL"],
  "gap_skills": ["Docker", "Testing"],
  "gap_severity": "minor",
  "career_alignment": "direct",
  "eligibility_status": "eligible"
}
```

AI Module can enrich the `summary` field with a more natural-language explanation if LLM is available. If not, the structured data alone is sufficient for the frontend to render a human-readable explanation.

---

17. AI Role and Explicit Limitations

17.1 What AI does in this system

AI task	Where	Input	Output	Why AI, not rules
Skill extraction from job postings	Data Pipeline	Raw opportunity description text	List of skill mentions	Unstructured text; too variable for regex
Skill normalization assistance	Data Pipeline	Raw skill mention, skill taxonomy	Matched canonical skill ID + confidence	Fuzzy matching problem; ML handles spelling/ synonym variation
Assessment question generation	Assessment Module	Skill ID, proficiency level, competency context	Draft MCQ question + 4 options + explanation	Reduces manual authoring effort; all output is human-reviewed

AI task	Where	Input	Output	Why AI, not rules
Personalized explanation generation	Recommendation Module	Structured match data (scores, gaps)	Natural language explanation	Converting structured data to readable prose
Semantic opportunity matching	Recommendation Module	Student profile embedding, opportunity description embedding	Similarity score	Handles cases where skill labels don't match but content does
Career domain explanation	Career Module (on-demand)	Domain name + industry context	Plain-language explanation	Converts structured data to student-friendly narrative

17.2 What AI does not do

Prohibited AI use	Reason
Define the skill taxonomy	AI does not have reliable knowledge of what skills a Backend Developer needs; this is expert-curated
Score assessments	Assessment scoring is deterministic comparison against answer keys
Make eligibility decisions	Deterministic rule-based check
Provide factual market statistics	AI may hallucinate; use scraped structured data
Store or retrieve student data	AI Module is stateless; it receives context from callers
Control security or authentication	Must be deterministic and auditable

Prohibited AI use	Reason
Autonomously modify the roadmap	Roadmap is rule-based; AI explains but does not decide

17.3 AI pipeline structure

No module calls an LLM directly. All calls pass through the AI Module:

```
Calling module (e.g., Recommendation)
          ↓
AI Module: validateInput()
          ↓
AI Module: buildPrompt(task, context)
          ↓
LLM API (OpenAI / Gemini)
          ↓
AI Module: parseResponse()
          ↓
AI Module: validateOutput()
          ↓
AI Module: fallback() if invalid
          ↓
Calling module receives validated result
```

17.4 Graceful degradation

If the AI Module cannot produce a valid response (LLM unavailable, response fails validation, rate limit exceeded):

- Recommendation explanation falls back to template-based structured explanation
- Skill extraction falls back to keyword matching against the skill taxonomy
- Assessment generation falls back to pre-authored questions in the question bank
- The system continues to function; AI failure is never a total system failure

17.5 Prototype-specific LLM configuration

> **Decision Pending** — Specific LLM provider to use.

> **Recommended default:** Google Gemini API (free tier for prototype; generous limits; no billing required for demo). OpenAI GPT-4o-mini is the backup if Gemini is insufficient. Both are callable via HTTP from the AI Module. The AI Module should abstract the provider so that switching requires only a configuration change.

---

18. Web Scraping and Data Ingestion Architecture

18.1 Architecture overview

The scraping pipeline is an **independent background process** — it is not part of the request-handling backend. It runs on a schedule and writes to the shared database.

```
Scheduler (cron / task queue)
         ↓
Source Connector (per-source)
         ↓
Raw Store (raw HTML or JSON, not parsed yet)
         ↓
Parser (per-source, extracts fields)
         ↓
Normalizer (maps to common schema)
         ↓
Skill Extractor (AI-assisted + keyword matching)
         ↓
Deduplicator (fingerprint comparison)
         ↓
Opportunity table (PostgreSQL)
```

18.2 Source connector design

Each source has its own connector because every website has a different structure. Connectors are isolated so that a change to one source's HTML structure only requires updating that source's connector.

```
connectors/
  unstop_connector.py
  internshala_connector.py
  aicte_connector.py
  manual_seed_loader.py
```

18.3 Scraping ethics and prototype approach

Source	Recommended prototype approach
Unstop	Review `robots.txt` and ToS. If scraping is permitted for non-commercial use, implement with rate limiting (≥ 3 seconds between requests). Otherwise use manually seeded data.
Internshala	Same as Unstop.
AICTE Portal	Government portal — public data. Check `robots.txt`. Apply rate limiting.
Manual seed	Always available. Team prepopulates 50–100 realistic opportunities for demo reliability.

> **The prototype must function even if all scrapers are disabled.** Manual seed data is the fallback for the demo.

18.4 Rate limiting and politeness

- Minimum 3-second delay between requests to any single source
- Respect `Crawl-delay` in `robots.txt` if specified
- Rotate between pages; do not hammer search endpoints
- Store `User-Agent` in configuration; use a descriptive value indicating this is a research/educational project
- Never run scrapers more than once per 24 hours per source in prototype mode

18.5 Error handling in pipeline

Error type	Handling
Source unreachable	Log, skip, retry at next scheduled run
Parse failure (unexpected HTML structure)	Log record, skip record, continue batch
AI extraction failure	Fall back to keyword matching, log degradation
Duplicate detected	Update `last_seen_at`, do not insert duplicate
Database write failure	Log, retry with exponential backoff (3 attempts)

---

19. Database Architecture and Major Entities

19.1 Database choice and rationale

Primary database: PostgreSQL

Reasons:

- Relational data (skills, prerequisites, user profiles) fits naturally in a relational model
- Complex queries (topological ordering of prerequisite graphs, join-heavy recommendation queries) are well-served by SQL
- PostgreSQL's JSON/JSONB columns handle semi-structured fields (raw eligibility text, evidence metadata) without requiring a separate document store
- Mature, well-documented, supported by all major hosting providers
- Team familiarity assumption

Secondary store: Redis

Purpose: Session token storage, caching frequent reads (domain list, skill graph nodes — these change rarely and are read on every page load)

File storage: Local filesystem for prototype; S3-compatible storage for production

> **No vector database in the prototype.** Semantic matching for the recommendation system

> will use the LLM's embedding capabilities through the AI Module without a dedicated vector store. If semantic search becomes a performance bottleneck at scale, introducing pgvector (a PostgreSQL extension) is the first upgrade step before a separate vector database.

19.2 Schema overview — key tables

The following are abbreviated schemas for the most important tables. Full column-level documentation is owned by the respective Role but is derived from these definitions.

`users`

```
id              UUID PRIMARY KEY
email           VARCHAR(255) UNIQUE NOT NULL
password_hash   VARCHAR(255) NOT NULL
created_at      TIMESTAMPTZ
updated_at      TIMESTAMPTZ
```

`student_profiles`

```
id              UUID PRIMARY KEY
user_id         UUID REFERENCES users(id)
full_name       VARCHAR(255)
institution     VARCHAR(255)
degree          VARCHAR(255)
year_of_study   INTEGER
interests       TEXT[]              -- array of interest tags
selected_role_id UUID REFERENCES roles(id)
created_at      TIMESTAMPTZ
updated_at      TIMESTAMPTZ
```

domains

```
id              UUID PRIMARY KEY
name            VARCHAR(255) UNIQUE
description     TEXT
demand_level    VARCHAR(50)         -- HIGH, MODERATE, EMERGING
is_active       BOOLEAN
display_order   INTEGER
```

roles

```
id              UUID PRIMARY KEY
domain_id       UUID REFERENCES domains(id)
name            VARCHAR(255)
description     TEXT
target_level    VARCHAR(50)         -- e.g., ENTRY, MID
is_active       BOOLEAN
```

skills

```
id              UUID PRIMARY KEY
name            VARCHAR(255) UNIQUE
description     TEXT
category        VARCHAR(255)        -- maps to competency
role_id         UUID REFERENCES roles(id)
target_proficiency VARCHAR(50)      -- AWARENESS through EXPERT
weight          FLOAT DEFAULT 1.0
is_active       BOOLEAN
```

skill_prerequisites

```
skill_id        UUID REFERENCES skills(id)
prerequisite_id UUID REFERENCES skills(id)
PRIMARY KEY (skill_id, prerequisite_id)
```

technology_branches

```
id              UUID PRIMARY KEY
parent_skill_id UUID REFERENCES skills(id)
name            VARCHAR(255)
description     TEXT
is_decision_point BOOLEAN
```

student_skill_states

```
id              UUID PRIMARY KEY
student_id      UUID REFERENCES student_profiles(id)
skill_id        UUID REFERENCES skills(id)
self_rating     VARCHAR(50)
assessed_level  VARCHAR(50)
assessment_score_pct FLOAT
confidence      VARCHAR(20)          -- HIGH, MEDIUM, LOW
evidence_boost  BOOLEAN DEFAULT FALSE
last_assessed_at TIMESTAMPTZ
UNIQUE (student_id, skill_id)
```

assessment_sessions

```
id              UUID PRIMARY KEY
student_id      UUID REFERENCES student_profiles(id)
role_id         UUID REFERENCES roles(id)
status          VARCHAR(50)          -- PENDING, IN_PROGRESS, COMPLETED
started_at      TIMESTAMPTZ
completed_at    TIMESTAMPTZ
total_questions INTEGER
```

questions

```
id              UUID PRIMARY KEY
skill_id        UUID REFERENCES skills(id)
proficiency_level VARCHAR(50)
question_text   TEXT
options         JSONB               -- [{label, text, is_correct}]
explanation     TEXT
difficulty      VARCHAR(50)
estimated_seconds INTEGER
source          VARCHAR(50)         -- MANUAL, AI_GENERATED
is_active       BOOLEAN
```

question_responses

```
id              UUID PRIMARY KEY
session_id      UUID REFERENCES assessment_sessions(id)
question_id     UUID REFERENCES questions(id)
selected_option INTEGER
is_correct      BOOLEAN
time_taken_secs INTEGER
```

roadmap_states

```
id              UUID PRIMARY KEY
student_id      UUID REFERENCES student_profiles(id)
role_id         UUID REFERENCES roles(id)
generated_at    TIMESTAMPTZ
last_updated_at TIMESTAMPTZ
current_milestone_id UUID          -- nullable FK to milestones
readiness_pct   FLOAT
```

milestones

```
id              UUID PRIMARY KEY
roadmap_state_id UUID REFERENCES roadmap_states(id)
skill_id        UUID REFERENCES skills(id)
sequence_order  INTEGER
status          VARCHAR(50)         -- NOT_STARTED, IN_PROGRESS, COMPLETED, SKIPPED, BRANC
branch_chosen   UUID REFERENCES technology_branches(id)
```

opportunities

```
id              UUID PRIMARY KEY
external_id     VARCHAR(512)
source          VARCHAR(50)         -- UNSTOP, INTERNSHALA, AICTE, MANUAL
original_url    TEXT
title           TEXT
organization    TEXT
type            VARCHAR(50)         -- INTERNSHIP, HACKATHON, COMPETITION, PROJECT, TRAINI
mode            VARCHAR(50)
location        TEXT
deadline        DATE
stipend         TEXT
eligibility_raw TEXT
description_raw TEXT
is_active       BOOLEAN DEFAULT TRUE
extracted_at    TIMESTAMPTZ
last_seen_at    TIMESTAMPTZ
fingerprint     VARCHAR(512) UNIQUE
```

opportunity_skill_tags

```
opportunity_id  UUID REFERENCES opportunities(id)
skill_id        UUID REFERENCES skills(id)
raw_mention     TEXT
confidence      FLOAT
PRIMARY KEY (opportunity_id, skill_id)
```

recommendations

```
id              UUID PRIMARY KEY
student_id      UUID REFERENCES student_profiles(id)
opportunity_id  UUID REFERENCES opportunities(id)
compatibility_score FLOAT
skill_match_score   FLOAT
career_alignment_score FLOAT
eligibility_score   FLOAT
interest_score      FLOAT
segment         VARCHAR(50)         -- READY_NOW, ALMOST_READY, ASPIRATIONAL
explanation_json JSONB
generated_at    TIMESTAMPTZ
```

resources

```
id              UUID PRIMARY KEY
skill_id        UUID REFERENCES skills(id)
title           TEXT
url             TEXT
type            VARCHAR(50)         -- VIDEO, ARTICLE, COURSE, DOCUMENTATION
platform        TEXT
is_free         BOOLEAN
estimated_hours FLOAT
```

evidence_items

```
id              UUID PRIMARY KEY
student_id      UUID REFERENCES student_profiles(id)
skill_id        UUID REFERENCES skills(id)
type            VARCHAR(50)         -- GITHUB, CERTIFICATE, PROJECT, COMPETITION
title           TEXT
url             TEXT
description     TEXT
submitted_at    TIMESTAMPTZ
verified        BOOLEAN DEFAULT FALSE
```

---

20. API Architecture and Major API Boundaries

20.1 API design principles

- **RESTful resource-oriented design** with standard HTTP verbs
- **JSON request and response bodies** throughout
- **JWT in Authorization header** on all authenticated endpoints
- **Versioned from the start:** all endpoints under `/api/v1/`
- **Consistent error response format** (see Section 23)
- **No chatty APIs** — prefer endpoints that return complete objects rather than requiring 5 round trips to render one screen

20.2 API boundary map

Each module owns its API surface. No two modules expose routes for the same resource.

```
/api/v1/
  auth/           → Auth Module
  profile/        → Profile Module
  careers/        → Career Module
  skills/         → Skill Graph Module
  assessments/    → Assessment Module
  roadmap/        → Roadmap Module
  opportunities/  → Opportunity Module
  recommendations/ → Recommendation Module
  resources/      → Resource Module
  portfolio/      → Portfolio Module
```

20.3 Major API endpoints

Auth

Method	Path	Description
POST	`/api/v1/auth/register`	Create account
POST	`/api/v1/auth/login`	Authenticate, receive JWT

Method	Path	Description
POST	`/api/v1/auth/refresh`	Refresh JWT
POST	`/api/v1/auth/logout`	Invalidate session

Profile

Method	Path	Description
GET	`/api/v1/profile/me`	Get current student profile
PUT	`/api/v1/profile/me`	Update profile fields
GET	`/api/v1/profile/me/skills`	Get full skill state
GET	`/api/v1/profile/me/dashboard`	Get aggregated dashboard data
POST	`/api/v1/profile/me/career`	Set/update career selection

Careers and Skill Graph

Method	Path	Description
GET	`/api/v1/careers/domains`	List all domains with demand signals
GET	`/api/v1/careers/domains/:id`	Domain detail and career paths
GET	`/api/v1/careers/roles`	List roles (filterable by domain)
GET	`/api/v1/careers/roles/:id`	Role detail including skill graph
GET	`/api/v1/skills/:id`	Skill detail with prerequisites
GET	`/api/v1/skills/graph`	Full skill graph for a role

Assessment

Method	Path	Description
POST	`/api/v1/assessments/self`	Submit self-assessment ratings

Method	Path	Description
POST	`/api/v1/assessments/start`	Start targeted assessment session
GET	`/api/v1/assessments/:session_id`	Get current session questions
POST	`/api/v1/assessments/:session_id/submit`	Submit answers, receive results
GET	`/api/v1/assessments/history`	Student's assessment history

Roadmap

Method	Path	Description
GET	`/api/v1/roadmap`	Get student's current roadmap
POST	`/api/v1/roadmap/generate`	Trigger roadmap (re)generation
POST	`/api/v1/roadmap/branch`	Record branch decision
GET	`/api/v1/roadmap/milestones`	List milestones with status
PUT	`/api/v1/roadmap/milestones/:id`	Update milestone status

Opportunities

Method	Path	Description
GET	`/api/v1/opportunities`	Browse opportunity index (paginated, filtered)
GET	`/api/v1/opportunities/:id`	Opportunity detail

Recommendations

Method	Path	Description
GET	`/api/v1/recommendations/opportunities`	Get personalized ranked opportunities

Method	Path	Description
GET	`/api/v1/recommendations/resources`	Get recommended learning resources

Portfolio

Method	Path	Description
POST	`/api/v1/portfolio/evidence`	Submit evidence item
GET	`/api/v1/portfolio/evidence`	List student's evidence
DELETE	`/api/v1/portfolio/evidence/:id`	Remove evidence item

20.4 Standard response envelope

All API responses use this envelope:

```
{
  "success": true,
  "data": { ... },
  "meta": {
    "timestamp": "2025-01-01T00:00:00Z",
    "version": "1.0"
  }
}
```

Error responses:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human-readable description",
    "details": [ ... ]
  },
  "meta": {
    "timestamp": "2025-01-01T00:00:00Z"
  }
}
```

---

21. Authentication and Authorization

21.1 Authentication mechanism

JWT (JSON Web Tokens)

- On login, the server issues an `access_token` (short-lived, 15 minutes) and a `refresh_token` (longer-lived, 7 days)
- Access token is sent in the `Authorization: Bearer <token>` header on every authenticated request
- Refresh token is used to obtain a new access token without re-authentication
- Tokens are signed using HS256 with a server-side secret

Why JWT over session cookies for prototype:
Stateless authentication is simpler to implement with a separate frontend and backend. Sessions would require shared session storage. JWT allows easy addition of mobile or other clients later.

> **Design Concern:** JWT cannot be invalidated server-side without a blocklist. For the prototype, we accept this limitation. For production, implement a token blocklist in Redis for logout and compromised-account scenarios.

21.2 Authorization model

Two roles for the prototype:

Role	Description	Access
`STUDENT`	Registered student	Own profile, assessments, roadmap, recommendations
`ADMIN`	System administrator	Read/write to skill graph, question bank, opportunity seeding

Authorization is enforced at the API layer. Middleware checks the JWT role claim before routing to the handler.

21.3 Data ownership enforcement

A student may only read and write their own profile, skill states, roadmap, assessments, and evidence. Any request for another student's data returns 403 Forbidden. The user ID is extracted from the JWT, not from a request body parameter.

---

22. Security and Privacy Considerations

22.1 Passwords

- Stored as bcrypt hashes (minimum cost factor 12)
- Plain text passwords never logged or stored anywhere

22.2 Input validation

- All API inputs validated against defined schemas before reaching business logic
- String fields sanitized to prevent XSS injection in stored data
- URL fields validated to be valid HTTP/HTTPS URLs before storage
- Database queries use parameterized statements; no string interpolation into SQL

22.3 Scraped data

- Treated as untrusted external input at all times
- Passed through the same validation and sanitization pipeline as user input
- Never executed or rendered as HTML

22.4 Environment secrets

- All API keys, database credentials, JWT secrets stored in environment variables
- Never committed to version control
- `.env.example` committed with placeholder values; `.env` in `.gitignore`

22.5 Student data

- Student skill profiles and assessment results are private to the student
- No student data is shared with external platforms
- Opportunity source links are provided for navigation; no student data is transmitted to external sources

22.6 HTTPS

- All deployed instances (even prototype) must use HTTPS
- Development may use HTTP on localhost only

---

23. Error Handling and Reliability

23.1 Error categories

Category	HTTP Status	Example
Validation error	400	Missing required field, invalid format
Authentication error	401	Missing or expired JWT
Authorization error	403	Attempting to access another student's data

Category	HTTP Status	Example
Not found	404	Resource ID does not exist
Conflict	409	Duplicate registration email
Rate limit	429	Too many requests
Server error	500	Unhandled exception
Service unavailable	503	AI Module or external dependency down

23.2 Error handling principles

- Every error returns the standard error envelope (Section 20.4)
- Server errors (5xx) log full stack traces server-side; clients receive only a safe message
- Validation errors return the specific field(s) that failed and why
- AI Module errors are caught and downgraded to deterministic fallback — never propagated as 503 to the student

23.3 Logging

- Structured JSON logging at INFO and ERROR levels
- Every request logs: method, path, status code, response time, user ID (if authenticated)
- Errors log: stack trace, request context, user ID
- Scraping pipeline logs: source, records fetched, records inserted, errors

> **Decision Pending** — Log aggregation tool for prototype.
> **Recommended default:** Simple file-based logging with rotation for prototype. ELK stack or cloud logging for production.

---

24. Testing Strategy

24.1 Test types

Type	Scope	Tool
Unit tests	Individual functions and service methods	Jest (frontend) / pytest (backend)
Integration tests	API endpoints with real database	pytest + TestClient
Component tests	Frontend components in isolation	React Testing Library
End-to-end tests	Complete user flows in browser	Playwright or Cypress
Data tests	Skill graph integrity, prerequisite cycles	pytest (custom assertions)

24.2 Priority for prototype

1. **Unit tests** on: scoring algorithm, roadmap generation logic, compatibility score calculation, skill proficiency mapping
2. **Integration tests** on: all API endpoints (happy path + major error cases)
3. **Data integrity tests** on: skill graph (no cycles in prerequisite edges, all referenced skill IDs exist)
4. **Manual end-to-end** on: the golden demo path (Section 25)

Full automated E2E test suite is post-prototype scope.

24.3 Test data

- A canonical test student profile must be maintained for reproducible testing
- The skill graph seed data must be stable and version-controlled
- Assessment questions used in tests must be clearly marked as test fixtures

---

25. Development and Integration Strategy

25.1 Phase sequence

Phase	Focus	Deliverable
1 — Foundation	Repository, CI, database, authentication, frontend shell	Deployed skeleton; login works
2 — Skill Graph	Domain/role/skill/prerequisite data and APIs	Browsable career/skill graph
3 — Profile + Assessment	Onboarding, self-assessment, targeted assessment, scoring	Student has a skill profile
4 — Roadmap	Baseline paths, personalization, decision points, progress	Student sees their roadmap
5 — Opportunities	Scraping pipeline, manual seed, normalization, Opportunity Index	Opportunities in database
6 — Recommendation	Compatibility scoring, ranking, explanation	Student sees matched opportunities
7 — AI	Skill extraction, explanations, assessment generation	AI-enhanced interactions
8 — Integration + Demo	End-to-end golden path, polish, deployment	Demo-ready system

25.2 Integration checkpoints

At the end of each phase, the integration owner (Role 6) verifies that:

- New APIs match documented contracts
- Frontend has been updated for new API responses
- Database schema changes have migrations committed
- No previously passing tests are broken

---

26. Git and Collaboration Strategy

26.1 Branch structure

```
main
  └── develop
        ├── feature/role1-frontend-onboarding
        ├── feature/role1-frontend-dashboard
        ├── feature/role2-auth
        ├── feature/role2-profile-api
        ├── feature/role3-skill-graph-data
        ├── feature/role3-assessment-engine
        ├── feature/role4-scraper-unstop
        ├── feature/role4-opportunity-index
        ├── feature/role5-recommendation-engine
        ├── feature/role5-ai-module
        └── feature/role6-integration
```

26.2 Rules

- `main` contains only stable, integrated, demo-ready code
- `develop` is the integration branch; all feature branches target `develop`
- Feature branches are named `feature/role{N}-{short-description}`
- Pull requests require at least one reviewer before merging to `develop`
- No direct pushes to `main` or `develop`
- Database migrations are committed alongside the code change that requires them

26.3 Commit conventions

Format: `type(scope): message`

Types: `feat`, `fix`, `refactor`, `test`, `docs`, `chore`

Example: `feat(skill-graph): add prerequisite edge validation`

26.4 Shared contracts

API contracts and database schemas are owned by the module owner (Roles 2, 3, 4, 5) and reviewed by Role 6. A contract change that affects another role requires notifying that role before merging.

---

27. Prototype and MVP Boundaries

27.1 What the prototype must demonstrate

Must work	Notes
Student registration and login	Basic auth
Career domain browsing	At least 6 domains
Career/role selection	Full skill graph for at least 2 complete roles
Self-assessment	All skills for the selected role
Targeted assessment	At least 20 questions per role
Skill profile display	Visual skill state
Adaptive roadmap generation	With at least one decision point per role
Branch selection and roadmap recalculation	Core differentiator
Opportunity matching	At least 50–100 indexed opportunities
Ranked opportunity display with explanations	Core differentiator
Redirect to external opportunity URL	Required
AI-generated personalized explanation	At least for opportunity recommendations

27.2 Acceptable prototype simplifications

Simplification	Why acceptable
Manual seed data for opportunities	Avoids scraping reliability issues during demo
Limited to 2–3 fully complete career paths	Quality over quantity; demo needs depth not breadth
No real-time scraping during demo	Pipeline runs in background; data pre-populated
No mobile-responsive design	Desktop demo is sufficient for judges
No email verification	Simplifies onboarding for demo
Uniform skill weights (all 1.0)	Weighted priority is refinement, not core concept
Template-based AI explanations if LLM is slow	Graceful degradation; structure matters more than eloquence

---

28. Future Scope Boundaries

The following are explicitly **excluded from the prototype** but are valuable future directions:

Future feature	Rationale for exclusion
Academic institution integration (LMS/ERP)	Requires institutional partnerships and API agreements
Employer/recruiter portal	Different product track; not in prototype scope
Industry partner content curation	Requires external partnerships
Real-time job market data APIs	Commercial APIs; cost and access
Social/community features (peer study, mentorship)	Different product area
Coding assessment with sandboxed	Significant infrastructure

Future feature	Rationale for exclusion
execution	
Automated portfolio analysis (GitHub code analysis)	Significant ML infrastructure
Mobile application	Separate development track
Advanced ML recommendation model	Core value established deterministically first
Production-grade scraping at scale	Not required for demo
Multi-language support	Scope constraint
WCAG full accessibility audit	Post-prototype
Certifications and digital badges	Partnership requirement
Placement tracking and analytics	Institutional feature
Peer comparison and leaderboard	Requires user base

---

29. Architectural Decisions and Rationale

29.1 Decision: Modular monolith over microservices

Decision: Single deployable backend with internal module separation.

Rationale: Microservices require service discovery, inter-service networking, distributed tracing, independent CI/CD pipelines, and significant operational overhead. A six-person prototype team should not carry this cost. The module boundaries established now are clean enough to extract into services later. The value of microservices is operational independence — we do not yet need it.

29.2 Decision: Skill graph as a structured DAG in PostgreSQL

Decision: Skills and prerequisites are stored as relational data (nodes + edges), not embedded in application code or configuration files.

Rationale: A hard-coded skill list cannot be queried, cannot be traversed algorithmically for topological sorting, and cannot be updated without a deployment. A relational graph enables: topological sort for roadmap ordering, prerequisite validation, gap analysis, and runtime updates by administrators without code changes.

29.3 Decision: Deterministic scoring first, AI second

Decision: The compatibility score formula is explicit and deterministic. AI provides supplementary semantic matching and natural language explanation.

Rationale: A recommendation system that is a black box cannot be explained to a student or to SIH judges. The formula approach produces explainable, auditable results. AI improves the quality of matches in ambiguous cases but does not replace the formula. This also means the system functions correctly even when AI is unavailable.

29.4 Decision: Opportunity aggregation, not hosting

Decision: The system indexes and links to external opportunities rather than hosting them.

Rationale: Building a competing internship marketplace would take the project in the wrong direction and directly compete with established platforms that have years of listings, partnerships, and student trust. The differentiation is the intelligence layer — showing the *right* opportunities for *this* student *now*. Hosting the opportunities is not necessary for that value.

29.5 Decision: Assessment is mandatory, evidence is optional

Decision: The targeted assessment is a required step. Portfolio evidence is optional and supplementary.

Rationale: Requiring evidence submission creates barriers for students who do not have GitHub accounts or certifications — which may include many first-year students and students from under-

resourced institutions. The platform should serve these students equally. Assessment is the equalizer; evidence is the enhancer.

29.6 Decision: No vector database in prototype

Decision: Semantic matching uses LLM embeddings via API calls rather than a local vector store.

Rationale: A vector database (Pinecone, Weaviate, pgvector) requires data ingestion pipelines, index management, and additional operational complexity. For the prototype scale (hundreds of opportunities, not millions), embedding comparison can be done in memory or through the LLM API without a dedicated store. pgvector can be added as a PostgreSQL extension if needed without architectural change.

29.7 Decision: Career exploration before aptitude testing

Decision: Students explore career domains with explanations before taking any aptitude or personality assessment.

Rationale: A student cannot meaningfully express preferences between careers they do not understand. Presenting an aptitude test before domain exploration produces unreliable preference data. Domain exploration creates the informed context necessary for preference expression to be meaningful.

---

30. End-to-End Example

This section traces the complete system behavior for one specific student scenario. It demonstrates every major component interaction.

Scenario: Priya

> Priya is a second-year Computer Science student. She has basic Python knowledge from class. She has heard of machine learning but has not done anything with it. She arrives at the platform with no clear career plan.

Step 1 — Registration

Priya registers with her email and academic details. The Auth Module creates her user record and issues a JWT. The Profile Module creates her student profile.

```
POST /api/v1/auth/register
{
  "email": "priya@example.com",
  "password": "...",
  "full_name": "Priya Sharma",
  "institution": "VIT Chennai",
  "degree": "B.Tech CSE",
  "year_of_study": 2
}

→ 201 Created
{
  "data": {
    "access_token": "eyJ...",
    "refresh_token": "eyJ..."
  }
}
```

Step 2 — Career Discovery

The frontend requests the domain list.

```
GET /api/v1/careers/domains

→ 200 OK
{
  "data": {
    "domains": [
      {
        "id": "uuid-ai-ml",
        "name": "Data and AI",
        "description": "Data scientists and ML engineers build systems that learn...",
        "demand_level": "HIGH",
        "top_technologies": ["Python", "TensorFlow", "PyTorch", "SQL", "scikit-learn"]
      },
      {
        "id": "uuid-swe",
        "name": "Software Engineering",
        "demand_level": "HIGH",
        ...
      },
      ...
    ]
  }
}
```

Priya taps "Explore" on the Data and AI domain. She reads the explanation, sees the technology overview, and clicks on two curated resources. She also briefly explores Software Engineering.

She decides Data and AI interests her more.

Step 3 — Career Selection

Priya selects Domain: Data and AI → Role: Machine Learning Engineer.

```
POST /api/v1/profile/me/career
{
  "role_id": "uuid-ml-engineer"
}

→ 200 OK
{
  "data": {
    "selected_role": "Machine Learning Engineer",
    "skill_graph_preview": {
      "total_skills": 18,
      "competencies": [
        "Programming Foundations",
        "Mathematics and Statistics",
        "Core ML",
        "Data Engineering",
        "MLOps Basics"
      ]
    }
  }
}
```

Step 4 — Self-Assessment

The Assessment Module fetches the 18 skills required for the ML Engineer role and presents them to Priya.

Priya rates herself:

Skill	Self-rating
Python	Average
Programming fundamentals	Proficient
Linear algebra	Beginner
Statistics	Beginner

Skill	Self-rating
Calculus	Beginner
SQL	Not familiar
Data manipulation (pandas)	Not familiar
Data visualization	Not familiar
ML fundamentals	Beginner
scikit-learn	Not familiar
Neural networks	Beginner
TensorFlow/PyTorch	Not familiar
Feature engineering	Not familiar
Model evaluation	Not familiar
Git	Average
Docker	Not familiar
Cloud basics	Not familiar
MLOps	Not familiar

Step 5 — Targeted Assessment

Based on the self-assessment, the Assessment Module generates a 24-question session:

- Programming fundamentals (rated Proficient) → 5 questions at Proficient level
- Python (rated Average) → 4 questions at Intermediate level
- Git (rated Average) → 3 questions at Intermediate level
- Linear algebra (rated Beginner) → 3 questions at Beginner level
- Statistics (rated Beginner) → 3 questions at Beginner level
- ML fundamentals (rated Beginner) → 3 questions at Beginner level
- Neural networks (rated Beginner) → 3 questions at Beginner level

Skills rated "Not familiar" are not assessed (cannot meaningfully test undeclared knowledge).

Priya completes the assessment. Results:

Skill	Self-rating	Assessment score	Assigned proficiency
Programming fundamentals	Proficient	88%	PROFICIENT ✓
Python	Average	40%	BEGINNER ↓
Git	Average	72%	PROFICIENT ↑
Linear algebra	Beginner	22%	BEGINNER ✓
Statistics	Beginner	18%	AWARENESS ↓
ML fundamentals	Beginner	30%	BEGINNER ✓
Neural networks	Beginner	15%	AWARENESS ↓

The system surfaces discrepancies gently:

```
"Your Python assessment suggests your practical skills may be
at Beginner level rather than Average. Your roadmap has been
adjusted to include Python fundamentals before moving to
data manipulation."

"Your Git skills are stronger than you rated yourself —
we've fast-tracked that milestone."
```

Step 6 — Skill Profile Generated

Priya's Skill Dashboard now shows:

```
Machine Learning Engineer — Skill Profile

Completed / At Target:
  ✓ Programming Fundamentals      PROFICIENT
  ✓ Git                           PROFICIENT

In Progress / Started:
  ● Python                        BEGINNER    (target: PROFICIENT)
  ● Linear Algebra                 BEGINNER    (target: INTERMEDIATE)
  ● ML Fundamentals                BEGINNER    (target: PROFICIENT)

Not Started / Below Awareness:
  ○ Statistics                    AWARENESS   (target: INTERMEDIATE)
  ○ Neural Networks               AWARENESS   (target: INTERMEDIATE)
  ○ SQL                           (not assessed)
  ○ pandas                        (not assessed)
  ... (11 more)

Overall Readiness: 14%
```

Step 7 — Adaptive Roadmap Generated

The Roadmap Module runs topological sort on the skill prerequisite graph for the ML Engineer role, filters out completed skills (Programming Fundamentals, Git), and orders the remainder:

```
Personalized Roadmap — Priya — ML Engineer

Phase 1: Python and Programming Core
  [1] Python language (intermediate)     ← START HERE
  [2] Object-oriented Python

Phase 2: Mathematics Foundations
  [3] Linear Algebra (with resources)
  [4] Statistics and Probability
  [5] Calculus basics

Phase 3: Data Handling
  [6] SQL fundamentals
  [7] Data manipulation with pandas
  [8] Data visualization

Phase 4: Core ML
  [9] ML fundamentals (theory + practice)
  [10] scikit-learn
  [11] Feature engineering
  [12] Model evaluation and metrics

                    ┌── DECISION POINT ──────────────────────────┐
                    │  Choose your framework focus:              │
                    │  → [TensorFlow] OR [PyTorch]               │
                    └────────────────────────────────────────────┘

Phase 5: Deep Learning (after branch choice)
  [13] Neural networks fundamentals
  [14] Selected framework
  [15] Model deployment basics

Phase 6: MLOps Basics
  [16] Docker
  [17] Cloud basics (AWS/GCP/Azure)
  [18] ML pipeline basics
```

Step 8 — Priya Chooses a Branch

At the decision point, Priya selects PyTorch. The Roadmap Module recalculates Phase 5:

```
Phase 5 (PyTorch branch):
  [13] Neural networks fundamentals
  [14] PyTorch tensors and autograd
  [15] Building networks in PyTorch
  [16] Model training loop
  [17] Model deployment with TorchServe (simplified)
```

The roadmap is updated. The frontend re-renders the roadmap with the PyTorch branch active.

Step 9 — Progress (time passes)

Over the next few weeks, Priya:

- Completes the Python milestone (takes an assessment, scores 78% → PROFICIENT)
- Completes SQL fundamentals (scores 65% → INTERMEDIATE)
- Submits a GitHub link to a pandas project as evidence

Her readiness updates: 14% → 31%

Step 10 — Opportunity Matching

Priya requests her recommendations:

```
GET /api/v1/recommendations/opportunities

→ 200 OK
{
  "data": {
    "ready_now": [
      {
        "opportunity_id": "uuid-hackathon-1",
        "title": "AI for Good Hackathon",
        "organization": "Unstop x NASSCOM",
        "type": "HACKATHON",
        "source": "UNSTOP",
        "original_url": "https://unstop.com/...",
        "compatibility_score": 0.78,
        "explanation": {
          "summary": "Strong match — your Python skills and ML fundamentals
                      are well-aligned. This is a beginner-friendly hackathon.",
          "matching_skills": ["Python", "ML Fundamentals"],
          "gap_skills": [],
          "career_alignment": "direct"
        }
      }
    ],
    "almost_ready": [
      {
        "opportunity_id": "uuid-internship-1",
        "title": "Junior ML Engineer Intern",
        "organization": "Bangalore Analytics Co.",
        "type": "INTERNSHIP",
        "compatibility_score": 0.61,
        "explanation": {
          "summary": "Close match — you meet most requirements.
                      Strengthening pandas and scikit-learn would make
                      you a strong candidate.",
          "matching_skills": ["Python", "SQL", "ML Fundamentals"],
          "gap_skills": ["pandas", "scikit-learn"],
          "gap_severity": "minor"
        }
      }
```

```
    ],
    "aspirational": [ ... ]
  }
}
```

Step 11 — Priya Reviews and Applies

Priya sees the hackathon is a strong match. She clicks "Apply". She is redirected to `https://unstop.com/...`. The platform's role is complete for this interaction.

She sees the internship is almost within reach. She notes the two gap skills (pandas, scikit-learn) and returns to her roadmap to continue progressing.

What this example demonstrates

System component	Demonstrated
Career discovery	Domain exploration before commitment
Skill graph	18-skill prerequisite-ordered graph for ML Engineer
Self-assessment	Student perception captured
Targeted assessment	Calibration against actual performance
Discrepancy handling	Python downgrade, Git upgrade
Adaptive roadmap	Topologically ordered, gap-filtered
Branch decision	PyTorch selected, roadmap recalculated
Progress tracking	Readiness 14% → 31%
Evidence submission	GitHub link as supplementary signal
Compatibility scoring	Explicit formula with explanations
Opportunity segmentation	Ready now vs almost ready vs aspirational

System component	Demonstrated
External redirect	Original URL preserved
AI explanation	Natural language summary from structured data

> **END OF STAGE 1 — GENERAL PROJECT DOCUMENTATION**

Waiting for your instruction.

Generate `Role 1 documentation` to begin individual role documents, or ask any clarification questions about the general documentation before proceeding.